

GIT

Git & GitHub

Tracking your work and showcasing it to the world

The version control every ML project needs

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A tool (Git) + a website (GitHub) for code
Git	Tracks every change to your project, locally
GitHub	Hosts your code online, for others to see
Why you need it	Every ML job wants a GitHub profile
Your repos	github.com/11shafayet — your five core projects

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Git vs GitHub	The tool vs the website
2	Why Version Control?	The problem it solves
3	The Git Workflow	Edit, stage, commit
4	Local and Remote	Push, pull, clone
5	Branches	Working without breaking things
6	Everyday Commands	The ones you'll use most
7	Git for ML Projects	Models, data, and .gitignore
8	A Strong GitHub Profile	What recruiters actually check
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. Git vs GitHub

These two names get mixed up constantly, so let's separate them clearly first. Git is a TOOL that tracks changes to your code over time, running on your own computer. GitHub is a WEBSITE that hosts Git projects online, so you can back them up, share them, and collaborate with others.

Git is the tool. GitHub is where you host it online.

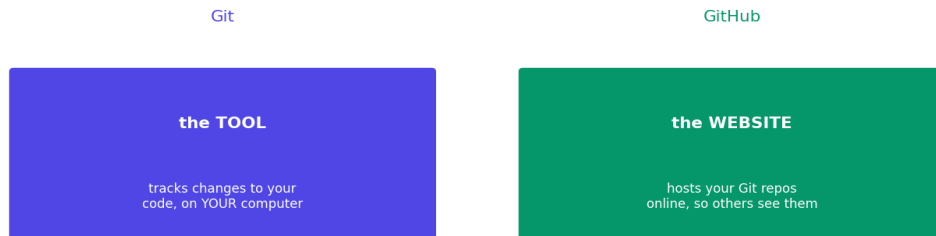


Fig 1 — Git is the tool that tracks changes. GitHub is the website that hosts your Git projects online.

You could use Git without ever touching GitHub — just tracking changes locally. But GitHub is where you'd put your work so recruiters, teammates, and the world can see it. For your MERN background this is likely familiar ground; this topic makes sure the fundamentals are airtight and adds the ML-specific habits.

2. Why Version Control?

Without Git, people save projects as folders like 'project_final', 'project_final_v2', 'project_ACTUALLY_final'. It's messy, error-prone, and you can't easily see what changed or undo a mistake. Git solves this by keeping a full history of every change, with the ability to go back to any earlier point at any time.

Beyond history, Git enables working safely: you can experiment on a copy (a branch) without touching the working version, and combine work from multiple sources (or multiple days of your own work) without losing anything. It's the backbone of virtually all professional software work, including ML projects.

■ *Even solo, Git is worth using for every project. It's your safety net — you can always go back to a working version if something breaks, and it's essentially required for any job, since ML roles expect a GitHub history of real work.*

3. The Git Workflow

Git has three areas your files move through. Understanding this flow makes every command make sense.

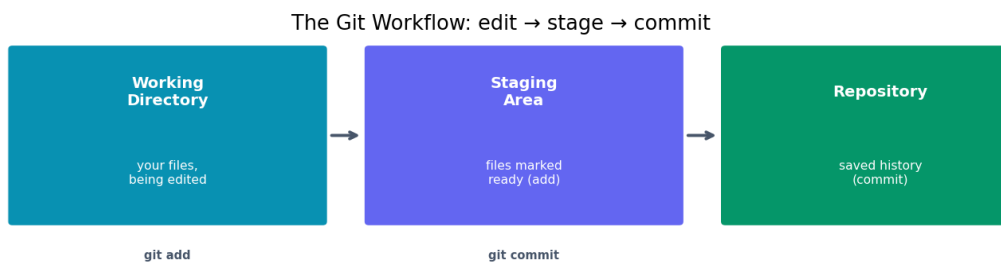


Fig 2 — Files move from your working directory, to staged, to committed — via add then commit.

```
# 1. You edit files (Working Directory)

# 2. Mark files as ready to save (Staging Area)
git add app.py
git add .           # stage everything changed

# 3. Save a snapshot with a message (Repository)
git commit -m 'Add prediction endpoint'
```

`git add` says 'I want this change included in my next save'. `git commit` actually saves that snapshot, with a message describing what changed. Staging exists so you can carefully choose exactly what goes into each commit, instead of saving everything blindly.

■ *Write clear commit messages — 'Add prediction endpoint' not 'update'. Good messages turn your commit history into a readable story of how the project was built, which is genuinely useful (and looks professional to anyone reviewing your repo).*

4. Local and Remote

So far everything lives on your computer (local). GitHub is a remote copy in the cloud. Three commands move work between the two.

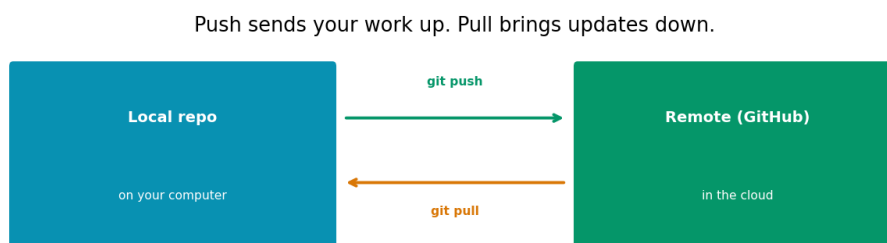


Fig 3 — Push sends your local commits up to GitHub. Pull brings updates from GitHub down to you.

Command	What it does
<code>git clone <url></code>	Download a copy of a GitHub repo to your computer

git push	Send your local commits up to GitHub
git pull	Bring the latest changes from GitHub down to you

clone is how you start working on an existing repo (yours or someone else's). push is how your local commits become visible on GitHub — without it, your work only exists on your own machine. pull keeps you in sync if the GitHub version has changes you don't have yet (from another device, or a collaborator).

5. Branches

A branch is a separate line of work, so you can try something new without touching your main, working code. The default branch is usually called main. You create a new branch for a feature or experiment, work there safely, and merge it back into main once it's ready.

Branching: work separately, then merge back

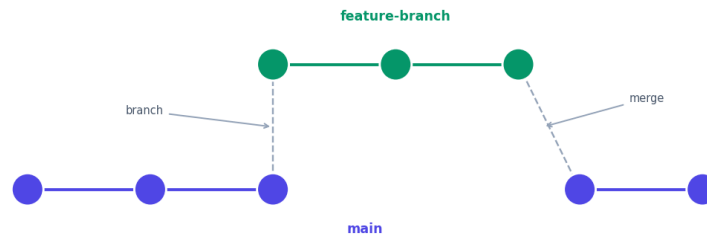


Fig 4 — Branching: work separately on 'feature-branch', then merge it back into 'main' when ready.

```
# Create and switch to a new branch
git checkout -b add-grad-cam

# ... make changes, add, commit as normal ...

# Switch back to main
git checkout main

# Bring the branch's changes into main
git merge add-grad-cam
```

■ Branches are especially useful when trying something risky — like a new model architecture or a major refactor. If it doesn't work out, you just don't merge it, and main stays clean. This habit alone prevents a lot of 'I broke my working project' moments.

6. Everyday Commands

A small set of commands covers most day-to-day Git use. Once these are second nature, you're covered for almost any project.

Command	What it does
git init	Start tracking a new project with Git
git status	See what's changed, staged, or not
git add <file>	Stage a file (or . for everything)
git commit -m 'msg'	Save a snapshot with a message
git log	See the commit history

<code>git diff</code>	See exactly what changed, line by line
<code>git checkout -b <name></code>	Create and switch to a new branch

7. Git for ML Projects

ML projects have a few extra habits worth knowing, because they often involve large files that Git wasn't originally designed for — model weights, datasets, checkpoints.

.gitignore — keep the mess out

A .gitignore file tells Git which files to NEVER track — typically things you don't want in your history: large datasets, model checkpoints, virtual environments, secret keys.

```
# .gitignore for a typical ML project
__pycache__/
*.pyc
venv/
.env
*.csv
*.pth
*.h5
checkpoints/
wandb/
```

■ *Never commit API keys or secrets (remember Topic 44). Put them in .env and add .env to .gitignore. If a key ever does get committed, treat it as compromised and revoke it — removing it from a later commit does NOT remove it from history.*

Large files — Git LFS

Git struggles with very large files (model weights, big datasets). Git LFS (Large File Storage) is an extension that handles big files properly, storing them efficiently instead of bloating your repo. For most portfolio projects, it's simpler to just not commit huge files at all — keep the training code, and describe how to download the data/weights in your README.

8. A Strong GitHub Profile

For your job search, GitHub isn't just storage — it's a public portfolio recruiters actually check. A few habits make a big difference in how your work is perceived.

Habit	Why it matters
Clear README per project	First thing anyone reads — explain what/why/how
Meaningful commit history	Shows real, ongoing work, not one big dump
Pinned repositories	Your best work is the first thing seen
Clean project structure	Signals professionalism
No secrets committed	Basic security hygiene, checked by some tools
Consistent activity	The green squares show you build regularly

You've already been doing much of this — ordering projects on your resume, updating your GitHub README, and keeping repos organised. Keep applying that same care per-repository: a short, clear README (what it does, key results, how to run it) turns a code dump into a piece of evidence an employer can quickly evaluate.

9. Common Mistakes

Mistake	Why it's a problem	Fix
Committing secrets/keys	Security risk, hard to fully remove	Use .env + .gitignore
No .gitignore	Repo bloated with junk files	Add one at project start
Vague commit messages	History becomes useless	Write clear, specific messages
One giant commit	Hard to review or undo	Commit in small, logical chunks
Committing huge model files	Slow, bloated repo	Use Git LFS or skip committing them
No README	Recruiters can't tell what it does	Always add a clear README

Quick Reference Cheatsheet

Task	Command
Start tracking	git init
Clone a repo	git clone <url>
Check status	git status
Stage a file	git add <file> / git add .
Commit	git commit -m 'message'
Push to GitHub	git push
Pull latest	git pull
See history	git log
See changes	git diff
New branch	git checkout -b <name>
Switch branch	git checkout <name>
Merge a branch	git merge <name>
Ignore files	add to .gitignore

The one idea to remember: Git tracks every change to your project locally through edit → stage → commit, and GitHub hosts that history online for the world to see. For ML work, add a .gitignore to keep large files and secrets out, and treat your README as part of the product — it's often the first thing a recruiter reads.

This is a bonus topic, extending your learning series beyond the original 44. Given your MERN background, much of this may be familiar — the ML-specific habits (.gitignore for models/data, avoiding committed secrets, strong READMEs) are the parts most worth reinforcing for your portfolio.